

TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

BOOK AI
Zadanie 2

2026

Oleksiy Orlyk
Oleksandr Semeniuk
Karyna Bubyr

Obsah

Úvod	4
1. Front-end.....	5
1.1. Prehľad.....	5
1.2. Analýza problému	5
1.3. Voľba technológií.....	6
1.4. Štruktúra aplikácie	6
1.5. Komunikácia s backendom	7
1.6. Priebeh aplikácie.....	7
1.7. Vstupy a výstupy	8
1.8. Spracovanie chýb.....	8
1.9. Správa stavu (State Management)	8
1.10. Štýlovanie.....	9
1.11. Spustenie aplikácie	9
1.12. Zhrnutie.....	10
2. Back-end.....	11
2.1. Prehľad.....	11
2.2. Analýza problému	11
2.3. Voľba technológií.....	11
2.4. Štruktúra backendu	12
2.5. API endpointy.....	14
2.6. Priebeh spracovania.....	14
2.7. Vstupy a výstupy	14
2.8. Spracovanie chýb.....	15
2.9. Zhrnutie.....	15
3. Implementácia databázy	16
3.1. Prehľad.....	16
3.2. Účel databázy.....	16

3.3.	Voľba technológie	16
3.4.	Štruktúra databázy.....	16
3.5.	Integrácia s backendom	18
3.6.	Tok dát	18
3.7.	Konfigurácia prostredia	19
3.8.	Spracovanie chýb	19
3.9.	Návrhové rozhodnutia	19
3.10.	Zhrnutie.....	20

Úvod

Cieľom tejto práce je návrh a implementácia webovej aplikácie využívajúcej moderné technológie a cloudové služby na riešenie zvolenej problematiky. Navrhnuté riešenie umožňuje používateľovi analyzovať knihu na základe fotografie jej obalu a následne získať relevantné informácie o danej knihe.

Aplikácia je navrhnutá ako viacvrstvový systém pozostávajúci z frontendu, backendu a databázovej vrstvy. Frontend zabezpečuje interakciu s používateľom a zobrazenie výsledkov, backend spracováva vstupné dáta a komunikuje s externými službami, zatiaľ čo databáza slúži na ukladanie a správu výsledkov analýzy.

Riešenie využíva viaceré cloudové služby od rôznych poskytovateľov. Na spracovanie obrázkov a generovanie textovej analýzy je použitá služba umelej inteligencie, zatiaľ čo na získanie informácií o knihách je využité externé API. Na ukladanie dát je použitá cloudová databáza.

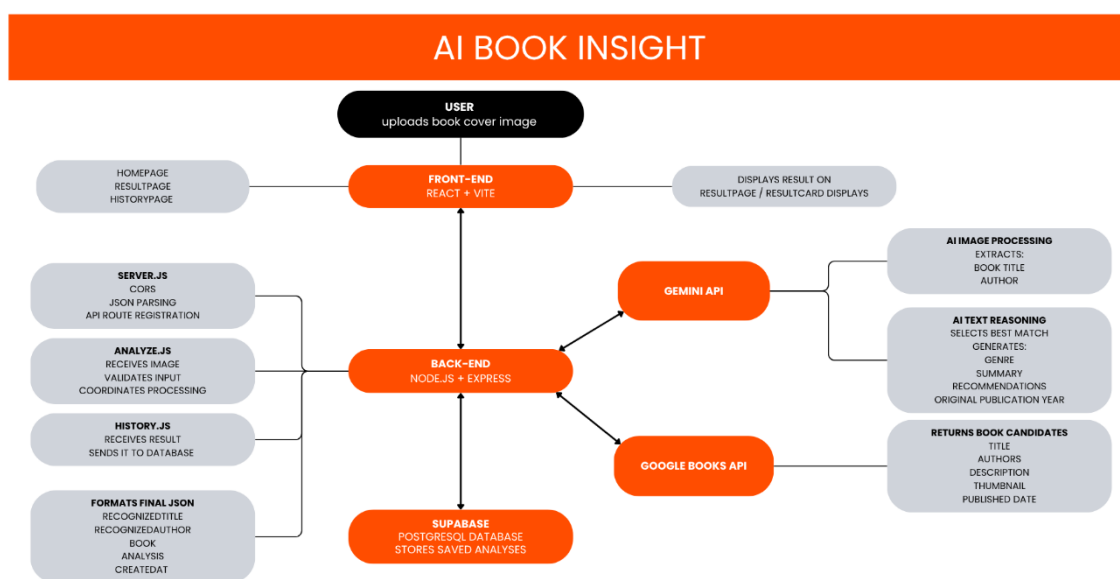
Hlavným cieľom aplikácie je prepojiť viacero technológií a služieb do jedného funkčného celku, ktorý poskytuje používateľovi jednoduchý a intuitívny spôsob získavania informácií o knihách. Zároveň riešenie demonštruje praktické využitie umelej inteligencie, REST API a cloudovej infraštruktúry v rámci modernej webovej aplikácie.

1. Front-end

1.1. Prehľad

Frontend aplikácie je implementovaný ako moderná jednostránková aplikácia (Single-Page Application) využívajúca knižnicu React a nástroj Vite. Poskytuje používateľské rozhranie pre interakciu so systémom, vrátane nahrávania obrázkov kníh, zobrazovania výsledkov analýzy a správy uloženej histórie.

Frontend komunikuje s backendom prostredníctvom REST API a je navrhnutý tak, aby bol modulárny, responzívny a používateľsky prívetivý.



Obr. 1. Štruktúra

1.2. Analýza problému

Hlavným cieľom frontendu je zabezpečiť jednoduché a intuitívne rozhranie pre používateľa, ktoré umožňuje:

- nahrávanie obrázka knihy,
- zobrazovanie výsledkov analýzy,
- ukladanie výsledkov,
- prehliadanie a správu uloženej histórie.

Aplikácia musí zároveň zvládať asynchrónnu komunikáciu s externými službami, zobrazovať stav načítavania (loading) a vhodne reagovať na vzniknuté chyby.

1.3. Voľba technológií

React

React bol zvolený z nasledovných dôvodov:

- podporuje komponentovú architektúru,
- zjednodušuje prácu so stavom aplikácie,
- umožňuje opätovné použitie UI komponentov,
- je vhodný pre dynamické webové aplikácie.

Vite

Vite je použitý pre:

- rýchly vývojový server,
- jednoduchú konfiguráciu,
- efektívne vytváranie produkčných buildov.

Axios / Fetch API

Používa sa na:

- komunikáciu s backendom,
- odosielanie požiadaviek a spracovanie odpovedí.

LocalStorage (záložné riešenie)

LocalStorage slúži ako záloha v prípade nedostupnosti backendu alebo databázy:

- umožňuje fungovanie aplikácie aj bez pripojenia k serveru,
- uchováva históriu používateľa lokálne.

1.4. Štruktúra aplikácie

Frontend je rozdelený na stránky a znovupoužiteľné komponenty.

Stránky

- HomePage.jsx - Hlavná vstupná stránka aplikácie, obsahuje rozhranie pre nahrávanie obrázka.
- ResultPage.jsx - Zobrazuje výsledok analýzy knihy.
- HistoryPage.jsx - Zobrazuje uložené výsledky a umožňuje ich odstránenie.

Komponenty

- Navbar.jsx - Slúži na navigáciu medzi jednotlivými stránkami.
- UploadBox.jsx - Zabezpečuje výber a nahranie obrázka.
- ImagePreview.jsx - Zobrazuje náhľad vybraného obrázka pred jeho odoslaním.
- LoadingSpinner.jsx - Zobrazuje indikátor načítavania počas spracovania požiadavky.
- ErrorMessage.jsx - Zobrazuje používateľsky zrozumiteľné chybové hlásenia.
- ResultCard.jsx - Zobrazuje výsledok analýzy knihy.
- BookCard.jsx - Znovupoužiteľný komponent na zobrazenie informácií o knihe.
- HistoryList.jsx - Zobrazuje zoznam uložených výsledkov.

1.5. Komunikácia s backendom

Všetka komunikácia s backendom je centralizovaná v súbore api.js.

Používané endpointy:

- POST /api/analyze
Odošle obrázok na spracovanie a prijme výsledok analýzy.
- POST /api/history
Uloží výsledok do databázy.
- GET /api/history
Načíta uložené výsledky.
- DELETE /api/history/:id
Odstráni konkrétny záznam z histórie.

Záložná logika

Ak backend nie je dostupný:

- ukladanie sa vykoná do localStorage,
- načítanie histórie sa realizuje z localStorage.

1.6. Priebeh aplikácie

- Používateľ otvorí hlavnú stránku.
- Nahrá obrázok pomocou komponentu UploadBox.
- Zobrazí sa náhľad obrázka.
- Používateľ odošle obrázok na spracovanie.
- Zobrazí sa indikátor načítavania.
- Backend spracuje požiadavku.
- Používateľ je presmerovaný na stránku s výsledkom.

- Zobrazí sa:
 - názov knihy,
 - autor,
 - informácie o knihe,
 - AI analýza.
- Používateľ môže výsledok uložiť.
- V histórii sa zobrazia uložené výsledky.

1.7. Vstupy a výstupy

Vstup

obrázok knihy (formát JPG, PNG)

Výstup

- rozpoznaný názov a autor,
- informácie o knihe,
- žáner,
- zhrnutie,
- odporúčania.

1.8. Spracovanie chýb

Frontend rieši chyby nasledovne:

- zobrazuje komponent ErrorMessage,
- zabraňuje pádu aplikácie,
- poskytuje používateľovi informácie o probléme.

1.9. Správa stavu (State Management)

Stav aplikácie predstavuje dynamické údaje, ktoré sa menia počas interakcie používateľa, napríklad vybraný súbor, stav načítavania, výsledky z API alebo uložená história.

V tejto aplikácii je stav spravovaný lokálne v jednotlivých React komponentoch pomocou hookov useState a useEffect.

Každý komponent si spravuje vlastný stav podľa potreby. Napríklad:

- komponent pre nahrávanie uchováva vybraný obrázok,
- hlavná stránka riadi stav načítavania a chýb,
- stránka výsledkov uchováva dáta získané z backendu,

- stránka histórie spravuje zoznam uložených výsledkov.

Dáta teda nie sú uložené v jednom centrálnom úložisku, ale každý komponent pracuje iba s tými údajmi, ktoré potrebuje.

Typy používaného stavu

- Stav nahrávania
Uchováva vybraný súbor pred jeho odoslaním.
- Stav načítavania
Informuje, či prebieha spracovanie požiadavky.
- Stav chyby
Uchováva informácie o vzniknutých chybách.
- Stav výsledku
Obsahuje výsledky analýzy knihy.
- Stav histórie
Uchováva uložené výsledky.

Dôvod použitia tohto prístupu

Tento prístup bol zvolený pretože:

- je jednoduchý a prehľadný,
- nevyžaduje použitie komplexných knižníc,
- je dostatočný pre rozsah aplikácie,
- zlepšuje udržiavateľnosť kódu.

1.10. Štýlovanie

Štýlovanie je realizované pomocou CSS:

- hlavný súbor main.css,
- responzívny dizajn,
- jednoduchý a prehľadný vzhľad.

1.11. Spustenie aplikácie

Inštalácia závislostí:

```
npm install
```

Spustenie vývojového servera:

npm run dev

1.12. Zhrnutie

Frontend aplikácie je navrhnutý ako modulárny a udržiavateľný systém založený na Reacte. Zabezpečuje používateľské rozhranie, komunikáciu s backendom a plynulý používateľský zážitok pomocou správneho riadenia stavu, načítavania a chýb.

Použitie znovupoužiteľných komponentov zlepšuje prehľadnosť a rozšíriteľnosť aplikácie, zatiaľ čo záložné mechanizmy zvyšujú jej spoľahlivosť aj v prípade nedostupnosti externých služieb.

2. Back-end

2.1. Prehľad

Backend aplikácie je implementovaný pomocou Node.js a frameworku Express. Predstavuje hlavnú spracovateľskú vrstvu systému, ktorá prijíma požiadavky z frontendu, komunikuje s externými API službami a vracia štruktúrované odpovede.

Backend je zodpovedný za spracovanie nahraných obrázkov, extrakciu informácií o knihe pomocou umelej inteligencie, získavanie dát o knihe z externých služieb a prípravu výsledkov na zobrazenie vo fronte.

Zároveň poskytuje endpointy na správu uloženej histórie, čím zabezpečuje prepojenie medzi frontendom a databázovou vrstvou.

2.2. Analýza problému

Hlavným cieľom backendu je spracovať vstup od používateľa (obrázok knihy) a transformovať ho na zmysluplné štruktúrované dáta.

Tento proces zahŕňa viacero krokov:

- prijatie a validáciu nahraného súboru,
- extrakciu relevantných informácií z obrázka,
- získanie zodpovedajúcich údajov o knihe z externých API,
- výber najvhodnejšieho výsledku,
- generovanie dodatočnej analýzy,
- vytvorenie jednotnej odpovede pre frontend.

Backend musí zároveň zvládať asynchrónne operácie, koordinovať komunikáciu s viacerými externými službami a zabezpečiť spoľahlivé spracovanie chýb.

2.3. Voľba technológií

Node.js

Node.js bol zvolený pretože:

- umožňuje vytvárať škálovateľné serverové aplikácie,
- efektívne pracuje s asynchrónnymi operáciami,
- dobre sa integruje s REST API službami.

Express

Express sa používa na:

- definovanie rout a API endpointov,
- spracovanie HTTP požiadaviek a odpovedí,
- organizáciu backend logiky prehľadným a modulárnym spôsobom.

Multer

Multer slúži na:

- spracovanie nahrávaných súborov z frontendu,
- prácu s multipart/form-data požiadavkami,
- uloženie obrázkov do pamäte pre ďalšie spracovanie.

Axios

Axios sa používa na:

- odosielanie HTTP požiadaviek na externé API,
- komunikáciu s Google Books API.

Google Gemini API

Používa sa na:

- extrakciu názvu knihy a autora z obrázka,
- výber najvhodnejšieho záznamu z výsledkov,
- generovanie analýzy (žáner, zhrnutie, odporúčania).

2.4. Štruktúra backendu

Backend je rozdelený do viacerých modulov, čo zlepšuje prehľadnosť a udržiavateľnosť.

Hlavný server

server.js

- inicializuje Express server,
- nastavuje middleware (CORS, spracovanie JSON),
- registruje API routy,
- spúšťa aplikáciu.

Routy

analyze.js

- obsluhuje endpoint /api/analyze,
- spracováva nahraný obrázok,
- integruje AI a externé API,
- vracia výsledok analýzy.

history.js

- obsluhuje endpoint /api/history,
- poskytuje funkcionality pre:
 - ukladanie výsledkov,
 - načítanie histórie,
 - mazanie záznamov.

Služby

geminiService.js

- zabezpečuje komunikáciu s Gemini API,
- vykonáva analýzu obrázka,
- generuje štruktúrované JSON odpovede.

booksService.js

- komunikuje s Google Books API,
- získava kandidátov na knihy,
- formátuje získané dáta.

Pomocné moduly

parseJson.js

- extrahuje validný JSON z odpovedí AI modelu,
- zabezpečuje bezpečné spracovanie výstupu.

Vrstva pripojenia k databáze

supabase.js

- inicializuje klienta databázy,
- zabezpečuje spojenie so službou Supabase.

Návrh databázy a jej konfigurácia sú popísané samostatne v časti venovanej databáze a deploymentu.

2.5. API endpointy

POST /api/analyze

Spracuje nahraný obrázok a vráti výsledok analýzy.

Postup:

1. prijatie obrázka z frontendu,
2. extrakcia názvu a autora pomocou AI,
3. dotaz na Google Books API,
4. výber najvhodnejšej knihy,
5. generovanie analýzy,
6. vrátenie štruktúrovanej odpovede.

GET /api/history

načíta uložené výsledky.

POST /api/history

uloží nový výsledok.

DELETE /api/history/:id

odstráni konkrétny záznam z histórie.

2.6. Priebeh spracovania

1. Frontend odošle obrázok na backend.
2. Backend prijme obrázok pomocou Multer.
3. Obrázok sa prevedie do Base64 formátu.
4. Gemini API extrahuje názov a autora.
5. Google Books API vráti zoznam kandidátov.
6. Gemini vyberie najvhodnejší výsledok.
7. Gemini vygeneruje dodatočnú analýzu.
8. Backend pripraví finálnu odpoveď.
9. Výsledok je odoslaný na frontend.

2.7. Vstupy a výstupy

Vstup

- obrázok (multipart/form-data)

Výstup

- názov knihy a autor,
- informácie o knihe (z Google Books),
- AI generované:
 - žáner,
 - zhrnutie,
 - odporúčania,
 - rok prvého vydania (ak je dostupný).

2.8. Spracovanie chýb

Backend rieši chyby nasledovne:

- validuje vstupné dáta,
- zachytáva chyby externých API,
- vracia vhodné HTTP status kódy,
- zabraňuje pádu aplikácie.

Príklady:

- chýbajúci obrázok → 400 Bad Request,
- nenájdená kniha → 404 Not Found,
- interná chyba → 500 Internal Server Error.

2.9. Zhrnutie

Backend je navrhnutý ako modulárny a škálovateľný systém, ktorý zabezpečuje spracovanie dát, integráciu externých API a komunikáciu s frontendom.

Zabezpečuje efektívne vykonávanie komplexných operácií, ako je analýza obrázka a agregácia dát, pričom frontend dostáva čisté, štruktúrované a konzistentné odpovede.

Rozdelenie logiky do rout, služieb a pomocných modulov zlepšuje udržiavateľnosť systému a umožňuje jeho jednoduché rozšírenie.

3. Implementácia databázy

3.1. Prehľad

Databázová vrstva aplikácie je implementovaná pomocou služby Supabase, ktorá poskytuje cloudovo hostovanú databázu PostgreSQL spolu s API na prácu s uloženými dátami.

Databáza slúži na perzistentné ukladanie dát generovaných používateľom, konkrétne histórie analyzovaných kníh. Umožňuje ukladať, načítavať a spravovať výsledky aj po opätovnom otvorení aplikácie.

Táto vrstva je navrhnutá ako nezávislá od backend logiky, pričom backend vystupuje iba ako sprostredkovateľ komunikácie medzi aplikáciou a databázou.

3.2. Účel databázy

Hlavným účelom databázy je:

- ukladať výsledky analýzy kníh,
- umožniť načítanie uloženej histórie,
- podporovať mazanie uložených záznamov,
- zabezpečiť perzistenciu dát aj po ukončení relácie používateľa.

Bez použitia databázy by aplikácia musela využívať iba lokálne úložisko (localStorage), ktoré je obmedzené na konkrétne zariadenie používateľa.

3.3. Voľba technológie

Supabase

Supabase bol zvolený z nasledovných dôvodov:

- poskytuje plne spravovanú databázu PostgreSQL,
- obsahuje vstavané REST API pre prácu s dátami,
- jednoducho sa integruje s JavaScript aplikáciami,
- ponúka bezplatný cloudový hosting vhodný pre vývoj a testovanie,
- zjednodušuje prístup k dátam a autentifikáciu (aj keď v tomto projekte sa autentifikácia nevyužíva).

3.4. Štruktúra databázy

Aplikácia využíva jednu hlavnú tabuľku:

Tabuľka: book_analyses

Táto tabuľka uchováva všetky relevantné informácie o analyzovaných knihách.

Hlavné stĺpce:

- id
Jedinečný identifikátor záznamu
- created_at
Čas vytvorenia záznamu
- recognized_title
Názov knihy rozpoznaný z obrázka
- recognized_author
Autor rozpoznaný z obrázka
- book_title
Finálny vybraný názov knihy
- book_authors
Zoznam autorov knihy
- google_books_id
Identifikátor z Google Books API
- published_date
Dátum vydania z Google Books
- display_published_year
Rok zobrazený používateľovi
- original_publication_year
Odhadovaný pôvodný rok vydania (generovaný AI)
- genre
Žáner knihy (generovaný AI)
- summary
Stručné zhrnutie (generované AI)
- recommendations
Odporúčané podobné knihy (generované AI)
- thumbnail
URL adresa obrázka obalu knihy
- description
Popis knihy z Google Books

- `selected_reason`
Dôvod výberu konkrétnej knihy
- `image_url`
Voliteľné pole pre uloženie referencie na obrázok
- `raw_data`
Kompletná JSON odpoveď uložená pre účely debugovania a rozšíriteľnosti

3.5. Integrácia s backendom

Databáza je prístupná prostredníctvom samostatného modulu:

`supabase.js`

Tento modul:

- inicializuje klienta Supabase,
- využíva environment premenné pre bezpečnú konfiguráciu,
- poskytuje opakovane použiteľné pripojenie k databáze.

Backend komunikuje s databázou prostredníctvom API rout definovaných v:

`history.js`

Tieto routy zabezpečujú:

- ukladanie nových záznamov (POST `/api/history`),
- načítanie záznamov (GET `/api/history`),
- mazanie záznamov (DELETE `/api/history/:id`).

3.6. Tok dát

Používateľ vykoná analýzu knihy

1. Backend vráti výsledok frontendu
2. Používateľ sa rozhodne uložiť výsledok
3. Frontend odošle dáta na endpoint `/api/history`
4. Backend vloží záznam do databázy
5. Dáta sú uložené a dostupné na ďalšie použitie
6. Stránka histórie si vyžiada dáta cez `/api/history`
7. Backend vráti uložené záznamy

3.7. Konfigurácia prostredia

Pripojenie k databáze vyžaduje nasledujúce environment premenné:

SUPABASE_URL=your_supabase_url

SUPABASE_ANON_KEY=your_supabase_key

Tieto hodnoty sa používajú na bezpečné pripojenie k Supabase projektu.

3.8. Spracovanie chýb

Databázová vrstva obsahuje základné mechanizmy na spracovanie chýb, ktoré zabezpečujú stabilitu a spoľahlivosť operácií s dátami.

Implementované sú:

- kontrola pripojenia k databáze (overenie inicializácie klienta),
- spracovanie chýb pri ukladaní dát,
- spracovanie chýb pri načítavaní dát,
- spracovanie chýb pri mazaní dát,
- vracanie štruktúrovaných chybových odpovedí backendu.

Príklady:

- chýbajúca konfigurácia → databázová funkcionálnosť je deaktivovaná a môže sa použiť fallback
- chyba pri ukladaní → dáta sa neuložia a backend vráti chybu
- chyba pri načítaní → žiadne dáta sa nevrátia
- chyba pri mazaní → záznam zostáva nezmenený

3.9. Návrhové rozhodnutia

Databázová vrstva je zámerne oddelená od backend logiky.

Toto oddelenie zabezpečuje:

- jasné rozdelenie zodpovedností,
- jednoduchšiu údržbu a ladenie,
- lepšiu škálovateľnosť do budúcnosti,
- flexibilitu pri zmene alebo výmene databázy bez zásahu do backendu.

Návrh zároveň podporuje:

- ukládanie štruktúrovaných aj pološtruktúrovaných dát (napr. raw_data vo formáte JSON),

- postupné rozširovanie schémy bez narušenia existujúcej funkcionality,
- efektívne vyhľadávanie v historických záznamoch.

Tento modulárny prístup je v súlade s modernými princípmi návrhu webových aplikácií.

3.10. Zhrnutie

Implementácia databázy poskytuje spoľahlivé a škálovateľné riešenie pre ukladanie dát o analyzovaných knihách pomocou služby Supabase.

Umožňuje trvalé uchovávanie výsledkov používateľa, efektívne načítanie a správu histórie a zároveň sa jednoducho integruje s backendom prostredníctvom definovaných API endpointov.

Oddelením frontendu, backendu a databázovej vrstvy vzniká modulárny, udržiavateľný a rozšíriteľný systém vhodný pre ďalší vývoj.